



Piscine Rust: Module 06

Summary: Boolean logic gates and arithmetic circuits in Rust.

Version: 1.00

Contents

I	General Rules	3
II	Project Structure	5
III	Part 1 : Basic Gates	6
IV	Part 2 : Derived Gates	8
V	Part 3 : Adders	10
VI	Part 4 : Multiplexer & Demultiplexer	12
VII	Turn in and Submission	14

Chapter I

General Rules

- Your program should not crash in any circumstances and should not quit unexpectedly. If it happens, your project will be considered non-functional and your grade will be 0.
- Your code must be written in Rust 1.93+
- You must compile your programs using `cargo run` and ensure there are no warnings or errors.
- Your programs must respect clippy linter standards.
- Functions names must match exactly what is requested.
- You must use **Cargo** for this module. Your project must be initialized with `cargo new`.
- Native logical operations implemented in Rust such as `&&`, `||` (both outside of if conditions), `^` are strictly forbidden.
- Only `&&` and `||` are allowed only in if conditions.

Cargo.toml Prerequisites

Your `Cargo.toml` must contain at minimum the following fields:

```
[package]
name = "Module 0X"
version = "x.x.x"
description = "Description"
edition = "20xx"

[dependencies]
```

- **name:** the name of your crate, must be `module_06`.
- **version:** semantic version of your project.
- **description:** a short description of the module.
- **edition:** must be set to 2024.
- No external dependencies are allowed.

Chapter II

Project Structure

This module is a single Cargo project organized into several source files:

```
module_06/  
  Cargo.toml  
  src/  
    main.rs  
    logical.rs  
    calc.rs
```

- `main.rs` — Entry point. Declares the modules, the `Bits` struct and the `Byte` type alias.
- `logical.rs` — Implements fundamental boolean logic gates.
- `calc.rs` — Implements arithmetic circuits and routing components built on top of the logic gates.

You must declare in `main.rs` the following public items:

```
pub mod calc;  
pub mod logical;  
  
#[derive(Debug)]  
pub struct Bits {  
    i0: bool,  
    i1: bool,  
    i2: bool,  
    i3: bool,  
}
```

The `Bits` struct represents 4 boolean inputs/outputs used by the multiplexer and demultiplexer.

Chapter III

Part 1 : Basic Gates



Part 1

Basic Gates: gate, not, and, or

File : `src/logical.rs`

Allowed: boolean operators in if conditions

Implement the following functions in `logical.rs`:

gate

The identity gate. Returns the input value unchanged.

```
pub fn gate(bit: bool) -> bool
```

Input	Output
false	false
true	true

not

Returns the logical negation of the input.

```
pub fn not(bit: bool) -> bool
```

Input	Output
false	true
true	false

and

Returns **true** if **both** inputs are **true**.

```
pub fn and(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	false
false	true	false
true	false	false
true	true	true

or

Returns **true** if **at least one** input is **true**.

```
pub fn or(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	false
false	true	true
true	false	true
true	true	true

Chapter IV

Part 2 : Derived Gates

	Part 2
Derived Gates: xor, nand, nor, xnor	
File : <code>src/logical.rs</code>	
Allowed: previously implemented gates only: <code>gate()</code> , <code>not()</code> , <code>and()</code> , <code>or()</code>	



These gates must be built **using only the gates from Part 1**. Do not use raw boolean operators directly.

xor

Exclusive OR. Returns `true` if exactly one input is `true`.

```
pub fn xor(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	false
false	true	true
true	false	true
true	true	false

nand

NOT AND. Returns the negation of the AND gate.

```
pub fn nand(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	true
false	true	true
true	false	true
true	true	false

nor

NOT OR. Returns the negation of the OR gate.

```
pub fn nor(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	true
false	true	false
true	false	false
true	true	false

xnor

Exclusive NOR. Returns the negation of the XOR gate.

```
pub fn xnor(bit1: bool, bit2: bool) -> bool
```

A	B	Output
false	false	true
false	true	false
true	false	false
true	true	true

Chapter V

Part 3 : Adders



Part 3

Arithmetic Circuits: `half_adder`, `full_adder`

File : `src/calc.rs`

Allowed: gates from `logical.rs`



These functions implement binary addition circuits using the logic gates defined in `logical.rs`.

`half_adder`

Takes two bits and returns a tuple (`carry`, `sum`).

```
pub fn half_adder(bit1: bool, bit2: bool) -> (bool, bool)
```

Half adder function return 2 variables:

- `sum`
- `carry`

A	B	Carry	Sum
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	false

full_adder

Takes three bits (two operands and a carry-in) and returns (carry, sum).

```
pub fn full_adder(a: bool, b: bool, c: bool) -> (bool, bool)
```

Full adder function return 2 variables:

- **sum**
- **carry**

A	B	C_in	Carry	Sum
false	false	false	false	false
false	false	true	false	true
false	true	false	false	true
false	true	true	true	false
true	false	false	false	true
true	false	true	true	false
true	true	false	true	false
true	true	true	true	true

Chapter VI

Part 4 : Multiplexer & Demultiplexer

	Part 4
Routing: multiplexer, demultiplexer	
File : <code>src/calc.rs</code>	
Allowed: gates from <code>logical.rs</code>	



These functions implement signal routing circuits. You may define private helper functions (e.g. a 3-input AND or a 4-input OR) inside `calc.rs`.

multiplexer

A 4-to-1 multiplexer. Selects one of the 4 inputs based on two selector bits `s0` and `s1`.

```
pub fn multiplexer(inputs: Bits, s0: bool, s1: bool) -> bool
```

The selection logic is:

- `s1=false, s0=false` → `inputs.i0`
- `s1=false, s0=true` → `inputs.i1`
- `s1=true, s0=false` → `inputs.i2`
- `s1=true, s0=true` → `inputs.i3`

Implementation hint: each output is a 3-input AND of the corresponding input and the appropriate combination of selector / negated selector bits. The final result is the 4-input OR of all four AND results.

demultiplexer

A 1-to-4 demultiplexer. Routes a single input bit to one of 4 outputs based on selector bits.

```
pub fn demultiplexer(bit: bool, s0: bool, s1: bool) -> Bits
```

Returns a Bits struct where only the selected output is set to the input value; all others are false.

- `s1=false, s0=false` → output on `i0`
- `s1=false, s0=true` → output on `i1`
- `s1=true, s0=false` → output on `i2`
- `s1=true, s0=true` → output on `i3`

Example:

```
let res = calc::demultiplexer(true, false, false);  
// Bits { i0: true, i1: false, i2: false, i3: false }
```

Chapter VII

Turn in and Submission

Turn in all your files in a Git repository as usual. All files in your repository will be evaluated. Double-check your file names and make sure all required files are present in the repository.



You must only turn in the files requested by the subject. Make sure you understand the Rust basics and be ready to correctly explain your code during evaluation.